

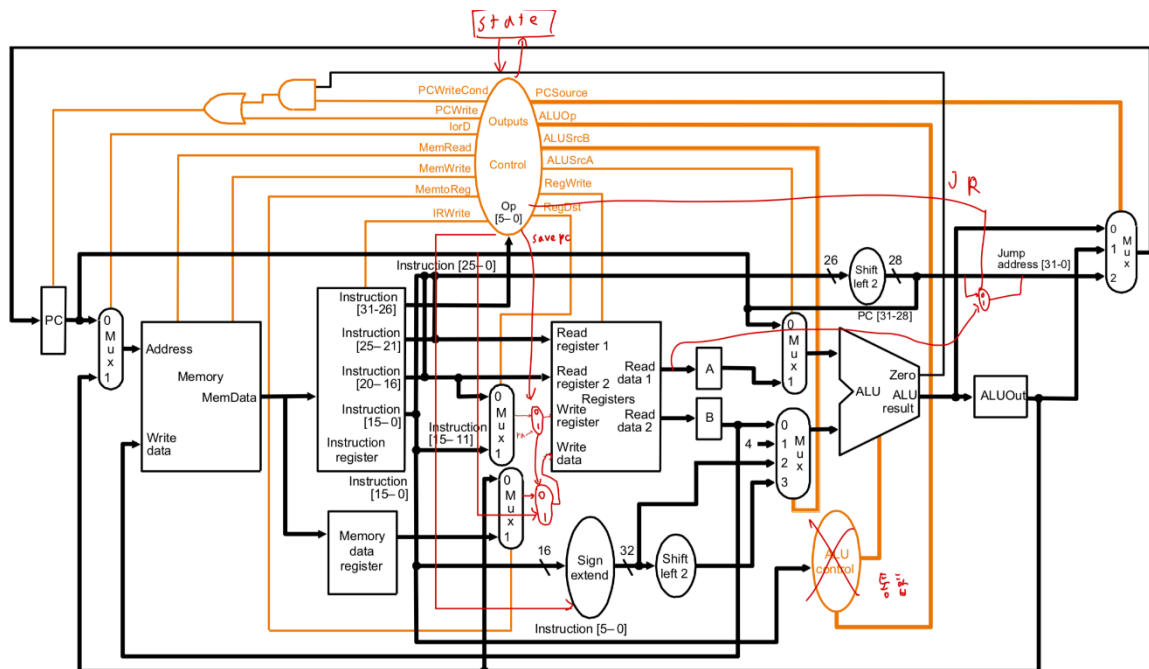
Lab4 Report

Name: 박수현

Student ID: 2022063672

Class Code: 12916

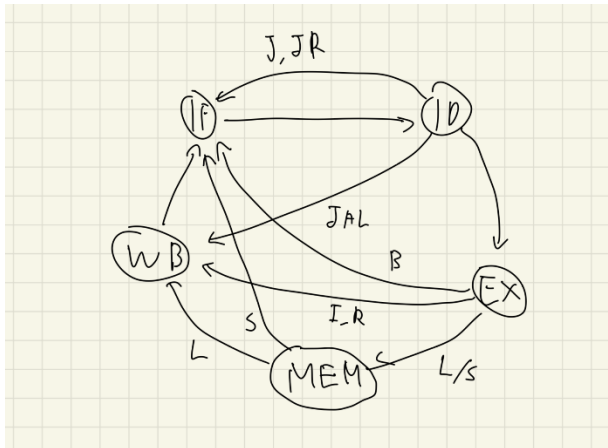
1. Overall Structure



HW3 와 비교하여 HW4에서 변경된 점은 다음과 같다.

- 멀티사이클 구현을 위하여 각 단계 사이에 이전 단계의 결과를 저장하는 레지스터 추가
- Instruction memory와 Data memory를 하나로 통합
- PC+4와 branch 연산을 수행하던 덧셈 모듈을 ALU로 통합
- 멀티사이클 CPU를 위한 컨트롤 비트 추가 & 삭제
- state 레지스터 추가

2. FSM



멀티사이클 CPU는 하나의 명령어를 5단계로 나누어 수행한다. 명령어는 요구되는 Register Transfer의 종류에 따라서, 수행하지 않는 단계를 건너뛸 수 있다. 그 절차를 위와 같은 FSM으로 나타낼 수 있으며, 각 단계별로 수행하는 작업들은 다음과 같다.

IF

- 메모리 모듈에서 PC값에 저장된 데이터를 꺼내와 Instruction register에 저장한다. 동시에 ALU를 사용하여 $PC + 4$ 값을 계산 후 저장한다.
- 모든 명령어는 ID state로 이동한다.

ID

- Instruction register에 저장된 명령어를 해독하여 명령어마다 필요한 작업을 수행한다.
 1. R-type: 레지스터에서 rs, rt 값을 찾아 A, B 레지스터에 저장한다
 2. I-type: A 레지스터에 rs 값을 저장하고, imm 값을 32비트로 확장한다.
 3. J: PC값을 $pc_upper \mid (target \ll 2)$ 로 변경한다.
 4. JR: PC값을 rs 레지스터에 저장된 값으로 변경한다.
 5. JAL: 현재 PC값을 ra 레지스터에 저장한다. 이후 WB 스테이지에서 PC값을 $pc_upper \mid (target \ll 2)$ 로 변경한다.
 6. B: 브랜치 타겟 계산하여 ALUOut 레지스터에 저장한다.

- ID 스테이지 수행이 완료된 경우, J, JR 명령어는 명령어 수행이 완료되었으므로 IF 스테이지로 이동한다. JAL 레지스터의 경우 PC값을 동시에 읽고 쓸 수 없어서 PC값 변경을 위해 WB 스테이지로 이동한다. 나머지 명령어들은 EX 스테이지로 이동한다.

EX

- ALUSrc 컨트롤 비트로 입력되는 피연산자들을 ALU 모듈에서 계산한다. 이 때, 브랜치 명령어는 이미 타겟 주소가 ALUout에 저장되어 있는 상태이므로, A와 B 레지스터의 비교 연산을 실행하고 그 결과에 따라 PC 값을 변경한다.

- EX 스테이지 수행이 완료된 경우, 브랜치 명령어는 IF 스테이지로 이동한다. LW/SW 명령어는 MEM 스테이지로 이동하고, I, R 타입 명령어는 WB 스테이지로 이동한다.

MEM

- LW의 경우 메모리에서 주소 위치에 저장된 데이터를 읽고 Memory data register에 저장한다. SW의 경우 입력되는 데이터를 메모리에 저장한다.

- LW 명령어는 Memory data register의 값을 범용 레지스터에 저장해야 하므로 WB 스테이지로 이동하고, SW 명령어는 IF 스테이지로 이동한다.

WB

- Write register에 Write data를 저장한다. JAL 명령어의 경우 Write register는 ra 레지스터이고, Write data는 현재 PC 값이다.

- 수행이 완료되었으면 IF 스테이지로 이동한다.

3. Implementation

- CTRL.v

구현의 편리함을 위해 always 블록 앞에서 모든 비트를 0으로 초기화한 다음, 각 케이스마다 필요한 비트를 설정한다.

케이스 블록 상단에는 FSM의 간선을 구현하고, 내부에서는 조건에 맞게 컨트롤 비트를 설정한다.

```
48     always @(*) begin
49         // 기본값 초기화
50         MemRead = 0;
51         MemtoReg = 0;
52         MemWrite = 0;
53         SignExtend = 0;
54         RegWrite = 0;
55         ALUOp = 0;
56         JR = 0;
57         SavePC = 0;
58         RegDst = 0;
59         ALUSrcA = 0;
60         ALUSrcB = 0;
61         PCSource = 0;
62         IRWrite = 0;
63         IorD = 0;
64         PCWrite = 0;
65         PCWriteCond = 0;
66
67         // state_next
68         case(state)
69             `ST_IF: begin
70                 state_next <= `ST_ID;
71
72                 // 명령어 불러오기 & PC+4
73                 IorD = 0;
74                 MemRead = 1;
75                 IRWrite = 1;
76                 PCWrite = 1;
77                 ALUOp = `ALU_ADDU;
78                 ALUSrcA = 0;
79                 ALUSrcB = 1;
```

```
63             `ST_ID: begin
64                 if(opcode == `OP_J || (opcode == 0 && funct == `FUNCT_JR)) state_next <= `ST_IF;
65                 else if(opcode == `OP_JAL) state_next <= `ST_WB;
66                 else state_next <= `ST_EX;
67
68                 // JAL
69                 // 현재 PC값 저장 You, 그저께 + ID 스테이지
70                 if(opcode == `OP_JAL) begin
71                     SavePC = 1;
72                     RegWrite = 1;
73                 end
74                 // J JR
75                 // PC값 변경
76                 else if(opcode == `OP_J || (opcode == 0 && funct == `FUNCT_JR)) begin
77                     PCWrite = 1;
78                     PCSource = 2;
79                     JR = opcode == 0 && funct == `FUNCT_JR;
80                 end
81                 else if(opcode == `OP_BEQ || opcode == `OP_BNE) begin
82                     ALUSrcA = 0;
83                     ALUSrcB = 3;
84                     ALUOp = `ALU_ADDU;
85                     SignExtend = 1; // 수정됨
86                 end
87                 // 일반적인 ID
88                 // BE에서 읽어오기 => 자동?
```

- CPU.v

3비트 state 레지스터와 멀티사이클을 위한 중간 레지스터를 추가한다. 이 레지스터들은 clock의 positive edge에서 입력된다.

```
65 // state 추가!!!!
66 reg [2:0] state;
67 wire [2:0] state_next; // (reg -> wire
68
69 // 멀티사이클을 위한 중간 레지스터
70 reg [31:0] inst;
71 reg [31:0] mem_data_reg;
72 reg [31:0] A;
73 reg [31:0] B;
74 reg [31:0] ALUOut; // You, 어제
75
154 // Update the Clock
155 always @(posedge clk) begin
156     if (rst) begin
157         PC <= 0;
158         state <= 0;
159     end
160     else begin
161         state <= state_next;
162         PC <= PC_next;
163         if (IRWrite) inst <= mem_read_data;
164         mem_data_reg <= mem_read_data;
165         A <= rd_data1;
166         B <= rd_data2;
167         ALUOut <= alu_result;
168     end
169 end
```

그 외의 구현은 위에서 제시한 회로도를 따르고 있다. JR 명령어의 경우, 기존 Jump address에 JR 컨트롤 비트를 입력으로 받는 2:1 MUX를 추가한 형태로 구현하였다. JAL 명령어의 경우 SavePC 컨트롤 비트를 입력으로 받는 2:1 MUX를 추가하여 SavePC가 set된 경우 주소는 ra레지스터, 입력 데이터는 PC값으로 설정 되도록 구현하였다.

```
126 // ALU-related wires
127 assign operand1 = ALUSrcA ? A : PC;
128 assign operand2 = (ALUSrcB <= 2'b01) ? (ALUSrcB == 2'b00 ? B : 4) : (ALUSrcB == 2'b10 ? ext_imm : ext_imm << 2);
129
130 always @(*) begin
131     wr_addr = SavePC ? 5'b11111 : (RegDst ? rd : rt);
132     // ALU 공유!!
133     wr_data = SavePC ? PC : (MemtoReg ? mem_data_reg : ALUOut);
134
135     PC_next = PC; // 이부분이 문제 traoubleshoot SuHyun, 1시간 전 * finish
136
137     if (PCWrite || (PCWriteCond && alu_result)) begin
138         case (PCSource)
139             0: PC_next = alu_result;
140             1: PC_next = ALUOut;
141             2: PC_next = JR ? rd_data1 : {(PC[31:28]), immj, 2'b00}; // 이부분??
142         endcase
143     end
144 end
```

- GLOBAL.v

State를 표현하는 5개의 상태를 추가하였다.

```
47 // state
48 `define ST_IF      3'd0
49 `define ST_ID      3'd1
50 `define ST_EX      3'd2
51 `define ST_MEM     3'd3
52 `define ST_WB      3'd4
```

4. Troubleshooting

CPU 모듈에서 매 사이클마다 $PC \leq PC_{next}$ 를 실행하도록 구현했으나, always @(*) 블록 안에서는 $PC_{next} = PC$ 구문을 추가하지 않아 PC가 변하지 않는 사이클에서 문제가 발생했다.